

```
---|---|---
 3 | 4 | 5
---|---|---
 6 | 7 | 8
```

Enter position 0 (X/O/-): 0
Enter position 1 (X/O/-): -
Enter position 2 (X/O/-): X
Enter position 3 (X/O/-): X
Enter position 4 (X/O/-): -
Enter position 5 (X/O/-): -
Enter position 6 (X/O/-): X
Enter position 7 (X/O/-): 0
Enter position 8 (X/O/-): 0

Your given board is:

```
 0 |   | X
---|---|---
 X |   | 
---|---|---
 X | 0 | 0
```

X count = 3
O count = 3

It is X's turn.

Enter your position (0-8): 1

After X's move:

```
 0 | X | X
---|---|---
 X |   | 
---|---|---
 X | 0 | 0
```

AI's turn (O)

AI selected position: 4

```
 0 | X | X
---|---|---
 X | 0 | 
---|---|---
 X | 0 | 0
```

O wins!

Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 b]

Enter "help" below or click "Help" above for more information.

>>>

RESTART

TIC-TAC-TOE

Enter the current board.

Use X for X, O for O and - for empty.

Positions:

```
0 | 1 | 2
---|---|---
3 | 4 | 5
---|---|---
6 | 7 | 8
```

Enter position 0 (X/O/-): O

Enter position 1 (X/O/-): -

Enter position 2 (X/O/-): X

Enter position 3 (X/O/-): X

Enter position 4 (X/O/-): -

Enter position 5 (X/O/-): -

Enter position 6 (X/O/-): X

Enter position 7 (X/O/-): O

Enter position 8 (X/O/-): O

Your given board is:

```
O |   | X
---|---|---
X |   | 
---|---|---
X | O | O
```

X count = 3

O count = 3

It is X's turn.

Enter your position (0-8): 1

After X's move:

```
O | X | X
---|---|---
X |   | 
---|---|---
X | O | O
```

```
if x_count == o_count:

    print("\nIt is X's turn.")

    player_move(board)

    print("\nAfter X's move:")
    print_board(board)

else:

    print("\nIt is O's turn.")

    ai_move(board)

    print("\nAfter AI's move:")
    print_board(board)

while True:

    if is_winner(board, 'X'):
        print("X wins!")
        break

    if is_winner(board, 'O'):
        print("O wins!")
        break

    if is_full(board):
        print("It's a draw!")
        break

    if board.count('X') == board.count('O'):

        print("Your turn (X)")
        player_move(board)

    else:

        print("AI's turn (O)")
        ai_move(board)

    print_board(board)
```

```
board = []
```

```
for i in range(9):
```

```
    while True:
```

```
        value = input("Enter position " + str(i) + " (X/O/-): ").upper()
```

```
        if value == 'X':
```

```
            board.append('X')
```

```
            break
```

```
        elif value == 'O':
```

```
            board.append('O')
```

```
            break
```

```
        elif value == '-':
```

```
            board.append(' ')
```

```
            break
```

```
        else:
```

```
            print("Please enter only X, O or -.")
```

```
print("\nYour given board is:")
```

```
print_board(board)
```

```
if is_winner(board, 'X'):
```

```
    print("X has already won!")
```

```
elif is_winner(board, 'O'):
```

```
    print("O has already won!")
```

```
elif is_full(board):
```

```
    print("The game is already a draw!")
```

```
else:
```

```
    x_count = board.count('X')
```

```
    o_count = board.count('O')
```

```
    print("X count =", x_count)
```

```
b[cell] = ' '
```

```
if score > best_score:
    best_score = score
    best_move = cell
```

```
b[best_move] = 'O'
```

```
print("AI selected position:", best_move)
```

```
def player_move(b):
```

```
    while True:
```

```
        try:
```

```
            position = int(input("Enter your position (0-8): "))
```

```
            if position < 0 or position > 8:
                print("Enter a number between 0 and 8.")
                continue
```

```
            if b[position] != ' ':
                print("Position already occupied.")
                continue
```

```
            b[position] = 'X'
            break
```

```
        except ValueError:
            print("Enter a valid number.")
```

```
print("TIC-TAC-TOE")
```

```
print()
```

```
print("Enter the current board.")
```

```
print("Use X for X, O for O and - for empty.")
```

```
print()
```

```
print("Positions:")
```

```
print(" 0 | 1 | 2 ")
```

```
print("---|---|---")
```

```
print(" 3 | 4 | 5 ")
```

```
print("---|---|---")
```

```
print(" 6 | 7 | 8 ")
```

```
board = []
```

```
if maximizing:
```

```
    best_score = -math.inf
```

```
    for cell in empty_cells(b):
```

```
        b[cell] = 'O'
```

```
        score = minimax(b, False)
```

```
        b[cell] = ' '
```

```
        best_score = max(best_score, score)
```

```
    return best_score
```

```
else:
```

```
    best_score = math.inf
```

```
    for cell in empty_cells(b):
```

```
        b[cell] = 'X'
```

```
        score = minimax(b, True)
```

```
        b[cell] = ' '
```

```
        best_score = min(best_score, score)
```

```
    return best_score
```

```
def ai_move(b):
```

```
    best_score = -math.inf
```

```
    best_move = None
```

```
    for cell in empty_cells(b):
```

```
        b[cell] = 'O'
```

```
        score = minimax(b, False)
```

```
        b[cell] = ' '
```

```
import math

def print_board(b):
    print()
    print(" {} | {} | {} ".format(b[0], b[1], b[2]))
    print("---|---|---")
    print(" {} | {} | {} ".format(b[3], b[4], b[5]))
    print("---|---|---")
    print(" {} | {} | {} ".format(b[6], b[7], b[8]))
    print()

def is_winner(b, player):

    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]

    for i, j, k in winning_positions:
        if b[i] == player and b[j] == player and b[k] == player:
            return True

    return False

def is_full(b):
    return ' ' not in b

def empty_cells(b):
    return [i for i in range(9) if b[i] == ' ']

def minimax(b, maximizing):

    if is_winner(b, 'O'):
        return 1

    if is_winner(b, 'X'):
        return -1

    if is_full(b):
        return 0

    if maximizing:
```


21/09/2026

1

Tic - Tac - Toe Implementation

Algorithm Tic-Tac-Toe (x, o)

- Step 1 : Start game
- Step 2 : Create empty 3x3 board
- Step 3 : Human play as x
- Step 4 : Computer play as o
- Step 5 : Human selects empty box
- Step 6 : Computer selects empty box
- Step 7 : check 3 same symbol in a row, column & diagonal
- Step 8 : If player win stop
else It's draw.
- Step 9 : Continue step 5 again
- Step 10 : stop.

21/9/26

Python 3.12.10 (tags/v3.12.10:0cc8128, A
Enter "help" below or click "Help" above

Enter vacuum location (A/B): A
Enter Room A (Clean/Dirty): Clean
Enter Room B (Clean/Dirty): Dirty

Initial State:

Location: A
Room A: Clean
Room B: Dirty
Room A is clean -> Move Right
Room B is dirty -> Suck

Final State:

Room A: Clean
Room B: Clean
Goal Achieved! Both rooms are clean.

Enter vacuum location (A/B): B
Enter Room A (Clean/Dirty): Dirty
Enter Room B (Clean/Dirty): Dirty

Initial State:

Location: B
Room A: Dirty
Room B: Dirty
Room B is dirty -> Suck
Room B is clean -> Move Left
Room A is dirty -> Suck

Final State:

Room A: Clean
Room B: Clean
Goal Achieved! Both rooms are clean.

```
# Vacuum Cleaner Agent
# Simple Reflex Agent + Goal Based Agent
```

```
location = input("Enter vacuum location (A/B): ").upper()
A = input("Enter Room A (Clean/Dirty): ").capitalize()
B = input("Enter Room B (Clean/Dirty): ").capitalize()
```

```
print("\nInitial State:")
print("Location:", location)
print("Room A:", A)
print("Room B:", B)
```

```
while A == "Dirty" or B == "Dirty":
```

```
    # Vacuum is in Room A
    if location == "A":
```

```
        if A == "Dirty":
            print("Room A is dirty -> Suck")
            A = "Clean"
```

```
        else:
            print("Room A is clean -> Move Right")
            location = "B"
```

```
    # Vacuum is in Room B
    else:
```

```
        if B == "Dirty":
            print("Room B is dirty -> Suck")
            B = "Clean"
```

```
        else:
            print("Room B is clean -> Move Left")
            location = "A"
```

```
print("\nFinal State:")
print("Room A:", A)
print("Room B:", B)
print("Goal Achieved! Both rooms are clean.")
```

29/09/2026

2. Vacuum Cleaner Implementation using Simple reflex agent.

Function REFLEX-VACUUM-AGENT [C location, status]
return an action

if status = Dirty then return pick Dirt

elseif location = A then return right.

elseif location = B then return left.

